

Trama: fluxos computacionais visuais e execução incremental em R

Pedro Mambelli Fernandes

João Paulo Assis Bonifácio

2026-09-19

Resumo

Apresenta-se o **Trama**, pacote R para construir, executar e inspecionar fluxos de computação em um editor visual. Cada operação é representada por um bloco com entradas, saídas e parâmetros declarados; seus resultados são mostrados no próprio diagrama. Os fluxos são persistidos em JSON e podem ser construídos por uma interface em R. O editor é renderizado em JavaScript com React; o Shiny transporta operações e eventos entre o navegador e o motor R. O plano de execução resolve dependências, calcula chaves de conteúdo e reutiliza artefatos já produzidos. Coleções instaláveis acrescentam operações estatísticas e formas de apresentação sem modificar o núcleo. Descrevem-se os contratos de tipos, catálogo, documento, execução e renderização, bem como um fluxo de transformação de dados incluído no repositório. A avaliação apresentada é documental e funcional: código, testes e exemplos verificam propriedades do sistema, mas não estimam ganho de desempenho nem efeito sobre a aprendizagem. O Trama oferece uma representação editável de análises em que resultados intermediários e dependências permanecem explícitos.

Palavras-chave: software estatístico; R; fluxo de dados; computação incremental; programação visual.

1 Introdução

Uma análise estatística reúne operações que raramente são independentes: a importação determina os dados disponíveis para limpeza, modelos dependem das variáveis produzidas nas transformações e gráficos dependem dos resultados anteriores. Em um script linear, essas relações ficam expressas pela ordem dos comandos e pelos objetos compartilhados. Em um **fluxo computacional**, elas são representadas diretamente por um grafo de operações e dependências. Essa representação permite identificar os insumos de cada resultado e selecionar as partes necessárias a uma nova execução. Sistemas como o Kepler estabeleceram esse modelo para compor e executar análises científicas em diferentes domínios (LUDÄSCHER et al., 2006).

A representação explícita não resolve, por si só, a reprodução de uma análise. Para obtê-la, é necessário conservar as entradas, os procedimentos e as condições de execução que influenciam cada resultado (SANDVE et al., 2013). No ambiente R, o pacote **targets** aplica essa ideia a pipelines declarativos: registra dependências e executa novamente os alvos afetados por mudanças (LANDAU, 2021). Ferramentas desse tipo atendem à automação da computação; a inspeção interativa dos resultados intermediários coloca uma exigência adicional sobre a forma de apresentação do fluxo.

O Trama foi desenvolvido nesse contexto. Ele reúne documento de fluxo editável, execução incremental e resultados visíveis em cada bloco. Este artigo tem como objetivo descrever sua representação, arquitetura de execução e extensão por coleções, e examinar as propriedades funcionais observáveis no código e nos exemplos distribuídos com o pacote.

2 Referencial e procedimento de exame

O Kepler organiza procedimentos científicos como redes de componentes conectados e separa a descrição do fluxo de sua execução (LUDÄSCHER et al., 2006). O pacote **targets** deriva dependências de funções e objetos R e usa o estado dessas dependências para decidir quais alvos atualizar (LANDAU, 2021). Os dois trabalhos mostram que a estrutura do grafo serve tanto à composição quanto ao planejamento computacional. Aqui, eles funcionam como referências de arquitetura; não se realizou uma comparação experimental com o Trama.

O aspecto visual impõe outra questão: o diagrama deve expor o encadeamento analítico sem ocultar o resultado produzido em cada etapa. McPhillips et al. (2009) discutem o desenho de fluxos por pesquisadores de domínio. O Trama adota blocos tipados e apresentações associadas a cada resultado para atender a essa tarefa. Shiny fornece a infraestrutura de aplicação web (CHANG et al., s.d.), e **ggplot2** oferece recursos de apresentação gráfica (WICKHAM, 2016). A utilidade da escolha visual para usuários ainda precisa ser avaliada empiricamente.

Examinou-se a versão 0.1.0 do pacote principal, as coleções, os testes e os projetos de exemplo presentes no repositório em 19 de setembro de 2026. Uma propriedade foi descrita como implementada quando apareceu na API ou no código e encontrou apoio em testes ou em um fluxo distribuído. O projeto **exemplos/vendas** serviu de caso para transformação de dados; os testes do núcleo foram usados para examinar edição, transporte e execução. Não se produziram medições de latência, memória ou usabilidade.

A arquitetura foi analisada por cinco contratos: tipo, catálogo, documento, execução e renderização. O tipo identifica valores e sua apresentação; o catálogo registra operações e adaptadores; o documento conserva blocos e arestas; o plano organiza a execução; os renderizadores JavaScript apresentam os artefatos. O critério adotado é funcional: as observações descrevem o estado do código examinado, sem inferir desempenho relativo ou validade estatística de todas as operações disponíveis.

3 Representação do fluxo

Um fluxo é um grafo dirigido cujos vértices são instâncias de blocos e cujas arestas ligam portas de saída a portas de entrada. Cada instância identifica o tipo de bloco, os parâmetros, a semente e uma identidade estável. As arestas registram os nomes das portas e, quando aplicável, a ordem de entradas múltiplas. O documento JSON guarda esses elementos separadamente da disposição visual. Alterar a posição de um card modifica a apresentação, sem mudar o cálculo previsto.

A Figura 1 apresenta um fluxo distribuído com o projeto. O mesmo documento pode ser editado no navegador ou construído por funções R. Essa equivalência permite versionar o fluxo em um formato textual e gerar fluxos programaticamente. O formato documenta a configuração de cada operação; a reexecução depende também das versões dos pacotes, dos arquivos externos e do ambiente R utilizado.

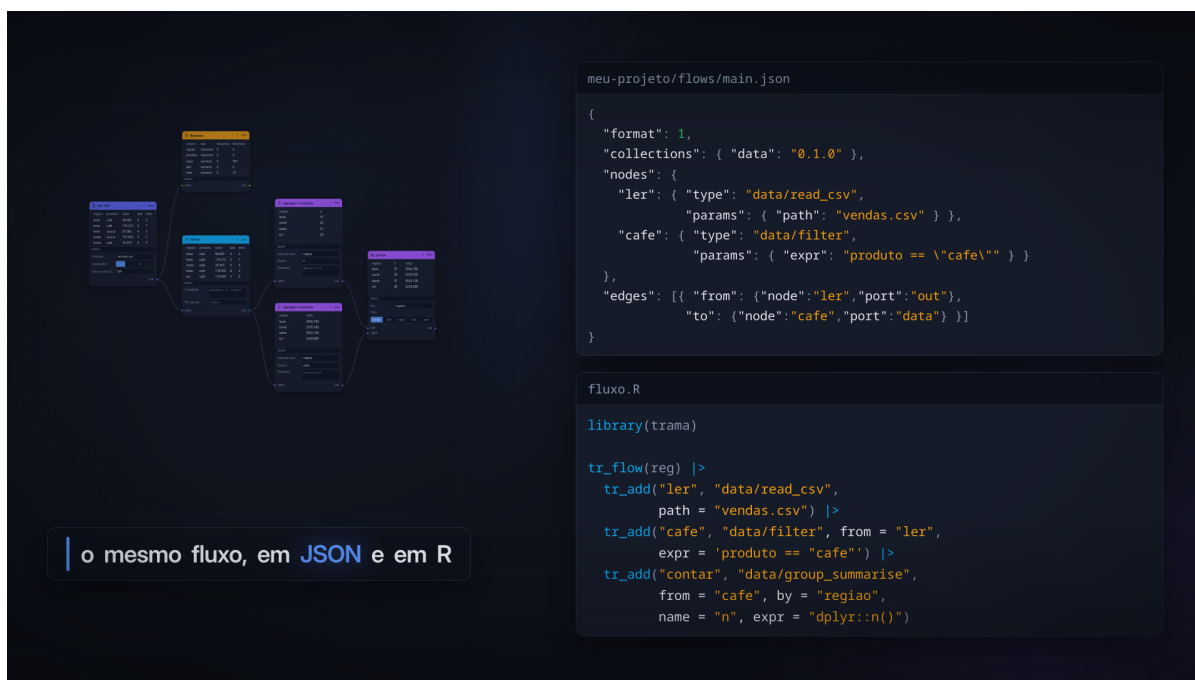


Figura 1: Representações do mesmo fluxo no editor, no documento JSON e na interface em R, conforme a documentação do Trama. Fonte: repositório do projeto.

As portas declaram tipo, obrigatoriedade e multiplicidade. O editor usa esses metadados para limitar ligações inválidas; o plano verifica a estrutura antes de despachar operações. Uma porta de entrada com múltiplas arestas conserva a ordem definida pelo índice da ligação. Quando os tipos não coincidem, um adaptador registrado para o par de tipos pode transformar o valor na aresta. O adaptador participa do cálculo sem acrescentar um bloco ao documento.

O documento contém tanto semântica quanto apresentação, mas apenas os campos semânticos participam das chaves de cache. Essa distinção evita recomputar uma análise quando se altera somente o tamanho ou a posição de um card. Ela também mantém a identidade de um bloco distinta de sua chave de execução: uma alteração de função pode invalidar o cache sem substituir a identidade documental do bloco.

4 Construção e edição

O projeto é iniciado por `tr_project()` e aberto com `tr_app()`. Na interface, seleciona-se um bloco na paleta, ligam-se portas e definem-se os parâmetros. Cada card recebe uma apresentação do resultado correspondente. O documento principal fica em `flows/main.json`, dentro da pasta do projeto. A aplicação usa Shiny para transportar comandos e eventos entre a interface e o processo R (CHANG et al., s.d.).

```
library(trama)
tr_app(tr_project(
  "meu-projeto",
  collections = c("trama.data", "trama.view")
))
```

Também se pode construir o documento por `tr_flow()`, `tr_add()` e `tr_link()`. `tr_add()` insere o bloco e pode conectar automaticamente a primeira entrada obrigatória livre e compatível; `tr_link()` explicita outras ligações, e `tr_set()` altera parâmetros após a criação. `tr_flow_code()` converte um documento de fluxo para código R. Essa interface serve à construção por script e à revisão textual de documentos gerados no editor.

O ciclo de edição distingue a revisão do documento da execução. Um comando altera o documento; o plano subsequente identifica as unidades necessárias aos alvos solicitados e consulta o armazenamento. As unidades cujo conteúdo já está disponível não são executadas novamente. Uma edição em parâmetro afeta o bloco correspondente e seus descendentes; os ramos independentes conservam seus artefatos. Essa propriedade aparece em testes do núcleo que alteram parâmetros e conferem a recomputação seletiva.

A edição visual conserva uma representação legível da análise, mas a legibilidade depende de nomes de blocos, parâmetros e arranjo definidos pelo autor. O sistema registra as dependências computacionais; ele não determina, por si só, se uma sequência de operações constitui um procedimento estatístico apropriado.

5 Planejamento e execução incremental

Para um alvo solicitado, `tr_plan()` percorre suas dependências e produz unidades em ordem topológica. A chave de cada unidade é calculada antes da execução por `rlang::hash()`. Ela incorpora a identificação e versão do bloco, a impressão digital da função, os parâmetros efetivos, as chaves das entradas, as impressões dos adaptadores e dos tipos de saída, e a impressão de recursos externos quando declarada. A semente entra para blocos estocásticos; blocos voláteis recebem um valor novo por plano. De forma abreviada,

$$k_v = H(t_v, q_v, F_v, p_v, s_v, K_v, A_v, T_v, e_v, n_v),$$

em que H é `rlang::hash`, F_v representa o código da função, K_v as chaves de entrada, A_v os adaptadores, T_v os tipos de saída, e_v o estado externo declarado e n_v o valor de volatilidade. Campos não pertinentes recebem valor nulo. Os parâmetros são normalizados antes do hash para que a gravação e a reabertura do JSON não mudem a chave apenas por diferenças de tipo ou formato.

Na instalação examinada, `rlang` 1.2.0 implementa `hash()` com XXH128, função não criptográfica de 128 bits. Ela atende à identificação rápida de conteúdo para o cache, não à autenticação dos artefatos. O Trama delega o algoritmo à dependência: uma atualização de `rlang` pode mudar os hashes e causar recomputação, conforme sua documentação (R-LIB, s.d.). A impressão da função cobre corpo, argumentos e referências internas da coleção até profundidade limitada; dependências externas do pacote não são rastreadas automaticamente.

O plano consulta o armazenamento por chave antes de despachar uma unidade. A Figura 2 mostra o efeito de alterar o filtro no fluxo de vendas: a leitura mantém sua chave, enquanto o filtro e seus descendentes recebem novas chaves. Testes do núcleo verificam o reuso em uma repetição sem alterações e a recomputação seletiva após editar um parâmetro. Não se mediu a economia de tempo para cargas de trabalho reais.

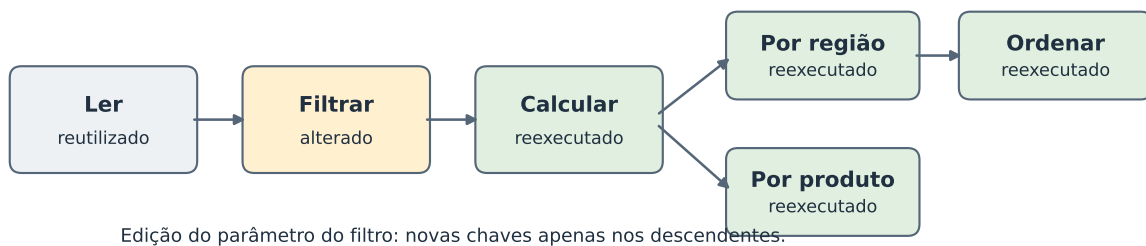


Figura 2: Recomputação seletiva após a alteração do parâmetro do filtro no fluxo de vendas. O bloco de leitura mantém sua chave; o filtro e seus descendentes recebem novas chaves. Fonte: elaboração a partir do fluxo distribuído com o Trama.

6 Interface, transporte e execução

A Figura 3 mostra a divisão da aplicação entre navegador e R. `tr_ui()` monta a página Shiny e publica os módulos JavaScript, as folhas de estilo e os arquivos das coleções. Um mapa de importações resolve React, React DOM, React Flow e o módulo `trama` a partir dos arquivos distribuídos com o pacote. Depois do carregamento das coleções, `editor.js` monta a aplicação React em `tr-root`. O React Flow desenha blocos, portas e arestas; o JavaScript administra a interação visual, enquanto o documento validado permanece no servidor R.

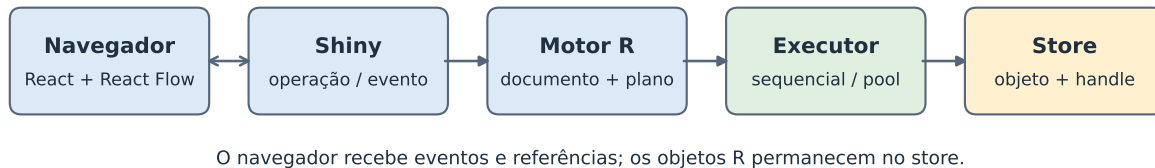


Figura 3: Divisão da arquitetura do Trama. React e React Flow desenharam o editor no navegador; o Shiny transporta operações e eventos; o motor R planeja, o executor computa e o armazenamento conserva objetos e handles. Fonte: elaboração a partir do código do pacote.

Uma edição gera uma operação enviada por `Shiny.setInputValue("tr_op", ...)` com número sequencial e revisão-base. `tr_server()` passa a operação a `tr_submit()`, que verifica a revisão, valida a mudança e atualiza o documento. O servidor devolve um evento `tr_event` por `sendCustomMessage()`. A interface recebe o eco da operação ou, quando necessário, o documento atualizado. Uma revisão defasada é recusada e o cliente recebe o documento corrente para ressincronização. A operação transportada é menor que o grafo completo e preserva a ordem das edições.

Após uma alteração semântica, o servidor calcula o plano e inicia o scheduler. Em execução sequencial, a função do bloco roda no processo R corrente; em pool, é despachada a workers. O armazenamento guarda o objeto e o handle de cada saída, e o servidor transmite eventos de estado ao navegador. O agendamento com `later` permite que a sessão Shiny processe novas mensagens entre passos do scheduler quando se usa o pool. Alterações apenas visuais não exigem recomputação.

O Shiny cumpre, portanto, o papel de transporte e coordenação da sessão. As regras de tipos, documentos, cache e execução estão em módulos R independentes da interface. A separação permite executar um fluxo sem abrir o editor e mantém os objetos R no armazenamento em vez de enviá-los integralmente ao navegador.

7 Coleções e contratos de extensão

Uma coleção é um pacote R que registra tipos, blocos, adaptadores, categorias e, quando necessário, código de apresentação. `tr_collection()` verifica a identificação da coleção e a pertença de seus elementos ao respectivo espaço de nomes. `tr_node()` associa uma função R a entradas, saídas, parâmetros, descrição e ajuda. `tr_type()` define o tratamento do valor produzido, inclusive sua apresentação.

A Tabela 1 resume as coleções distribuídas no repositório. O conteúdo é caracterizado pelas operações registradas; a tabela não atribui equivalência metodológica entre procedimentos de domínios diferentes.

Tabela 1: Coleções do Trama e operações predominantes no repositório, 2026. Fonte: descrições e código das coleções.

Coleção	Operações predominantes
<code>trama.data</code>	Importação, inspeção, transformação, agregação e gravação de dados
<code>trama.view</code>	Gráficos produzidos com <code>ggplot2</code>
<code>trama.series</code>	Operadores, decomposição, modelos e gráficos de séries temporais
<code>trama.models</code>	Ajuste, testes, diagnósticos e predição de modelos estatísticos
<code>trama.multi</code>	Procedimentos de análise multivariada
<code>trama.sampling</code>	Seleção, planejamento, estimação e avaliação amostral
<code>trama.ml</code>	Classificação, regressão, avaliação e extração de regras

Os renderizadores JavaScript recebem o artefato e o handle. A coleção `trama.view`, por exemplo, produz uma imagem para gráficos do `ggplot2`; o núcleo a apresenta pelo componente de imagem registrado. Uma coleção também pode registrar componentes React próprios por meio do módulo `trama` publicado no navegador. O mecanismo permite incorporar apresentações específicas sem alterar a biblioteca central de cards. O registro explícito de tipos e operações também fornece informações para a paleta e para a validação das conexões.

8 Fluxo de transformação de dados

O projeto `exemplos/vendas` parte de `vendas.csv`. O documento lê o arquivo, filtra linhas com `valor > 180`, calcula `receita = valor * qtd` e se divide em dois ramos: receita por região e receita por produto. O ramo regional também é ordenado por receita. Esses passos constam do JSON distribuído e podem ser reconstruídos pela interface R abaixo.

```
reg <- tr_registry()
tr_use("trama.data", registry = reg)

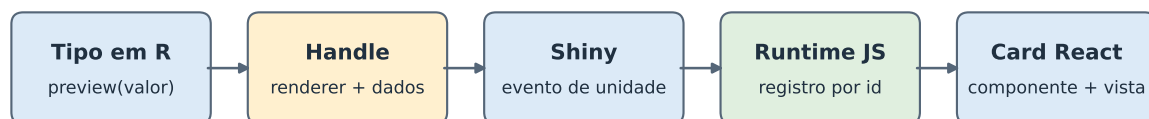
tr_flow(reg) |>
  tr_add("ler", "data/read_csv", path = "vendas.csv") |>
  tr_add("filtrar", "data/filter", expr = "valor > 180", from = "ler") |>
  tr_add("total", "data/mutate", name = "receita",
        expr = "valor * qtd", from = "filtrar") |>
  tr_add("porregiao", "data/group_summarise", by = "regiao",
        name = "receita_total", expr = "sum(receita)", from = "total") |>
  tr_add("porproduto", "data/group_summarise", by = "produto",
        name = "receita_total", expr = "sum(receita)", from = "total") |>
  tr_add("ranking", "data/arrange", cols = "receita_total",
        desc = TRUE, from = "porregiao")
```

A ramificação permite examinar um mesmo resultado intermediário sob duas agregações. Se a expressão do filtro for modificada, ambos os ramos dependentes deverão receber novas chaves; se apenas a ordenação for alterada, os resultados de agregação poderão ser reutilizados. Esse padrão decorre da estrutura do grafo e foi objeto de testes de recomputação seletiva no núcleo. A validade da receita calculada pressupõe que `valor` e `qtd` tenham os significados e unidades esperados no arquivo de entrada.

9 Renderização dos resultados

A execução de um bloco produz um valor R, que é armazenado segundo o contrato do respectivo tipo. A função `preview()` do tipo prepara uma representação para a interface e devolve `tr_preview(renderer, data, files)`. `data` contém informação reduzida que pode trafegar no evento; `files` identifica arquivos de apresentação, como imagens, gravados no armazenamento. O `handle` reúne a chave, o tipo, um resumo opcional e essa descrição da apresentação. O valor R completo permanece no armazenamento.

A Figura 4 mostra o caminho do valor até o card. O identificador `renderer` é resolvido em `runtime.js`, que mantém um registro de componentes React. O card recebe o artefato, o `handle` e uma função que transforma caminhos do armazenamento em URLs. Assim, o JavaScript escolhe como desenhar uma apresentação produzida em R sem depender da classe do objeto R. Um mesmo renderizador pode oferecer vistas alternativas para o mesmo artefato.



A apresentação é selecionada pelo identificador do renderer, sem enviar o objeto R.

Figura 4: Caminho de renderização de um resultado. O tipo R produz um preview, que é registrado em um handle; o evento do Shiny entrega sua referência; o runtime JavaScript escolhe pelo identificador o componente React que desenha o card. Fonte: elaboração a partir dos contratos de tipo e interface do Trama.

O núcleo registra apresentações gerais, como tabela, texto, imagem e erro. Uma coleção pode chamar `registerRenderer()` para acrescentar uma apresentação e `registerWidget()` para acrescentar um editor de parâmetros. Os módulos das coleções são carregados antes da montagem do editor, de modo que seus registros estejam disponíveis quando os cards forem criados. Em `trama.view`, o R gera um PNG do gráfico, enquanto um renderizador de imagem do núcleo o apresenta no navegador. O mesmo mecanismo aceita componentes React específicos de uma coleção.

A separação entre cálculo e apresentação limita o conteúdo enviado ao cliente e preserva extensões por pacote. Ela pressupõe que cada tipo forneça um preview adequado ao tamanho e à semântica do resultado; um preview reduzido não substitui o objeto completo armazenado para análises posteriores.

10 Protocolo de eventos e estados

O protocolo distingue comandos de edição de notificações da execução. Após montar a árvore React e registrar o receptor de mensagens, o navegador envia `tr_ready`; o servidor responde com catálogo, documento e estado inicial. Cada edição segue como `{seq, base_rev, op}` em `input$tr_op`. `seq` distingue operações sucessivas com conteúdo idêntico; `base_rev` vincula a edição à revisão vista pelo cliente. `tr_submit()` aceita ou recusa a operação, e o servidor responde com `op_applied` ou `op_rejected`. Em uma recusa, envia também o documento corrente para ressincronizar a interface.

A Tabela 2 apresenta os eventos relevantes quando uma execução é iniciada, seja na abertura do documento, seja após uma edição semântica. O scheduler emite estados de unidades com identificador de execução, bloco e chave. `tr_server()` os encaminha pelo canal `tr_event` como mensagens `unit`, preservando o estado original em `unit_type`. O evento `run_finished` encerra a execução. Mensagens de catálogo e documento compartilham o canal de saída, mas não representam estados do scheduler.

Tabela 2: Estados emitidos pelo scheduler do Trama e uso pela interface, 2026. Fonte: código de `R/scheduler.R`, `R/transport.R` e `inst/www/editor.js`.

Evento	Significado para a interface
<code>cached</code>	Artefato existente; o handle permite rerepresentar o resultado.
<code>running, progress, partial</code>	Unidade iniciada, fração publicada ou apresentação parcial.
<code>done</code>	Execução concluída; os handles das saídas estão disponíveis.
<code>failed, blocked, invalid</code>	Falha da unidade, dependência indisponível ou documento incompleto.
<code>cancelled, run_finished</code>	Trabalho interrompido ou execução encerrada.

O scheduler mantém conjuntos explícitos de unidades concluídas, pendentes e em execução. Um passo despacha unidades cujas dependências terminaram e coleta resultados; `later` agenda o próximo passo no servidor Shiny. Ao chegar uma edição que supera a execução corrente, unidades em curso cuja chave permanece no novo plano são adotadas; as demais são canceladas. O navegador atualiza o estado dos cards a partir dos eventos recebidos, sem receber os objetos R completos. Esse desenho torna visíveis cache, progresso e falhas. Os eventos descrevem o estado da sessão; o documento JSON e os artefatos armazenados permanecem como registros persistentes do fluxo. A associação entre revisão, chave da unidade e identificador da execução delimita a aplicação de cada evento.

11 Discussão e condições de validade

A separação entre documento, plano, armazenamento e interface permite alterar a forma de edição sem redefinir a execução. O protocolo de eventos torna estados do scheduler observáveis no editor sem inserir regras de apresentação no motor. Também permite que uma coleção acrescente operações e apresentações específicas, preservando os contratos do núcleo. O fluxo de vendas demonstra composição tabular e ramificação; os contratos de renderização permitem inspecionar os resultados em cada etapa. Esses resultados descrevem capacidades implementadas, condicionadas à instalação das coleções e dependências correspondentes.

O cache pressupõe que as entradas relevantes sejam refletidas na chave. Arquivos externos requerem impressão digital apropriada; mudanças em dependências de funções R podem requerer invalidação explícita. Funções com efeitos colaterais ou resultados dependentes de estado externo demandam atenção adicional, pois a reutilização por conteúdo é adequada apenas quando a chave representa as condições que determinam a saída. O Trama expõe recursos para esses casos, mas a autoria de uma coleção continua responsável por declarar corretamente o comportamento de seus blocos.

As coleções estatísticas disponibilizam procedimentos, mas a presença de um bloco não determina adequação de modelo, qualidade de dados ou validade inferencial. Uma análise de amostragem depende do desenho e de suas premissas; um modelo preditivo depende de validação e da forma de particionamento. O fluxo torna operações e resultados inspecionáveis, mas as decisões metodológicas permanecem com o analista.

Em relação a **targets**, o Trama compartilha a ideia de dependências explícitas e reexecução seletiva, mas organiza a autoria em torno de blocos editáveis no navegador e apresentações por resultado. A relação com Kepler está na composição visual de operações; o Trama concentra seu contrato de operações em pacotes R instaláveis. Essas diferenças são de arquitetura e interface, sem estabelecer superioridade de um sistema sobre outro (LANDAU, 2021; LUDÄSCHER et al., 2006).

Os testes e exemplos examinados não constituem estudo de desempenho, comparação com outras ferramentas ou avaliação com usuários. Portanto, não se conclui que a interface reduza tempo, erros ou esforço de programação. Uma avaliação futura poderia especificar tarefas, perfis de usuários e medidas antes da coleta, mas esses resultados não fazem parte deste artigo.

12 Conclusões

1. No estado do repositório examinado, o Trama representa fluxos em JSON e em R e os edita em uma interface visual, mantendo explícitos blocos, portas, parâmetros e dependências.
2. Para blocos cujas dependências estão representadas na chave calculada por `rlang::hash()`, o plano reutiliza artefatos existentes e restringe a recomputação aos descendentes de uma alteração.
3. Com as coleções instaladas, o núcleo executa operações de domínio, enquanto React apresenta resultados e o Shiny transporta operações e eventos entre editor e motor R.
4. Os exemplos e testes do repositório sustentam as propriedades funcionais descritas, mas não fornecem estimativas de desempenho, usabilidade ou efeito sobre resultados científicos.

13 Disponibilidade do código e dos dados

O código, os testes e os dados do fluxo de vendas estão disponíveis no [repositório público do Trama](#), sob licença MIT para o código do projeto. A versão examinada é a 0.1.0, conforme `DESCRIPTION`; para repetição exata, recomenda-se fixar também a revisão do repositório e as versões das dependências.

14 Declarações

Ética: a análise utiliza código e dados de exemplo distribuídos no repositório; não envolve participantes humanos. **Conflitos de interesse:** não informados pelos autores neste manuscrito. **Financiamento:** não informado neste manuscrito. **Contribuições:** os autores constam do arquivo `DESCRIPTION`; a distribuição por papéis CRediT depende de confirmação dos próprios autores. **Uso de IA:** este manuscrito foi redigido com assistência de IA e requer revisão e aprovação dos autores antes de submissão.

Referências

- CHANG, W. et al. *shiny: Web Application Framework for R*. Pacote R, s.d. Disponível em: <https://search.r-project.org/CRAN/refmans/shiny/help/shiny.html>. Acesso em: 19 set. 2026. {#ref-shiny}
- LANDAU, W. M. The targets R package: a dynamic Make-like function-oriented pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, v. 6, n. 57, p. 2959, 2021. DOI: [10.21105/joss.02959](https://doi.org/10.21105/joss.02959).
- LUDÄSCHER, B. et al. Scientific workflow management and the Kepler system. *Concurrency and Computation: Practice and Experience*, v. 18, n. 10, p. 1039–1065, 2006. DOI: [10.1002/cpe.994](https://doi.org/10.1002/cpe.994).
- MCPHILLIPS, T. et al. Scientific workflow design for mere mortals. *Future Generation Computer Systems*, v. 25, n. 5, p. 541–551, 2009. DOI: [10.1016/j.future.2008.06.013](https://doi.org/10.1016/j.future.2008.06.013).
- R-LIB. *Hashing: rlang reference*. Disponível em: <https://rlang.r-lib.org/reference/hash.html>. Acesso em: 19 set. 2026.
- SANDVE, G. K. et al. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, v. 9, n. 10, e1003285, 2013. DOI: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- WICKHAM, H. *ggplot2: Elegant Graphics for Data Analysis*. New York: Springer, 2016. Disponível em: <https://ggplot2.tidyverse.org/authors.html>. Acesso em: 19 set. 2026. {#ref-ggplot2}